Multiprocessing, Multithreading and SIMD

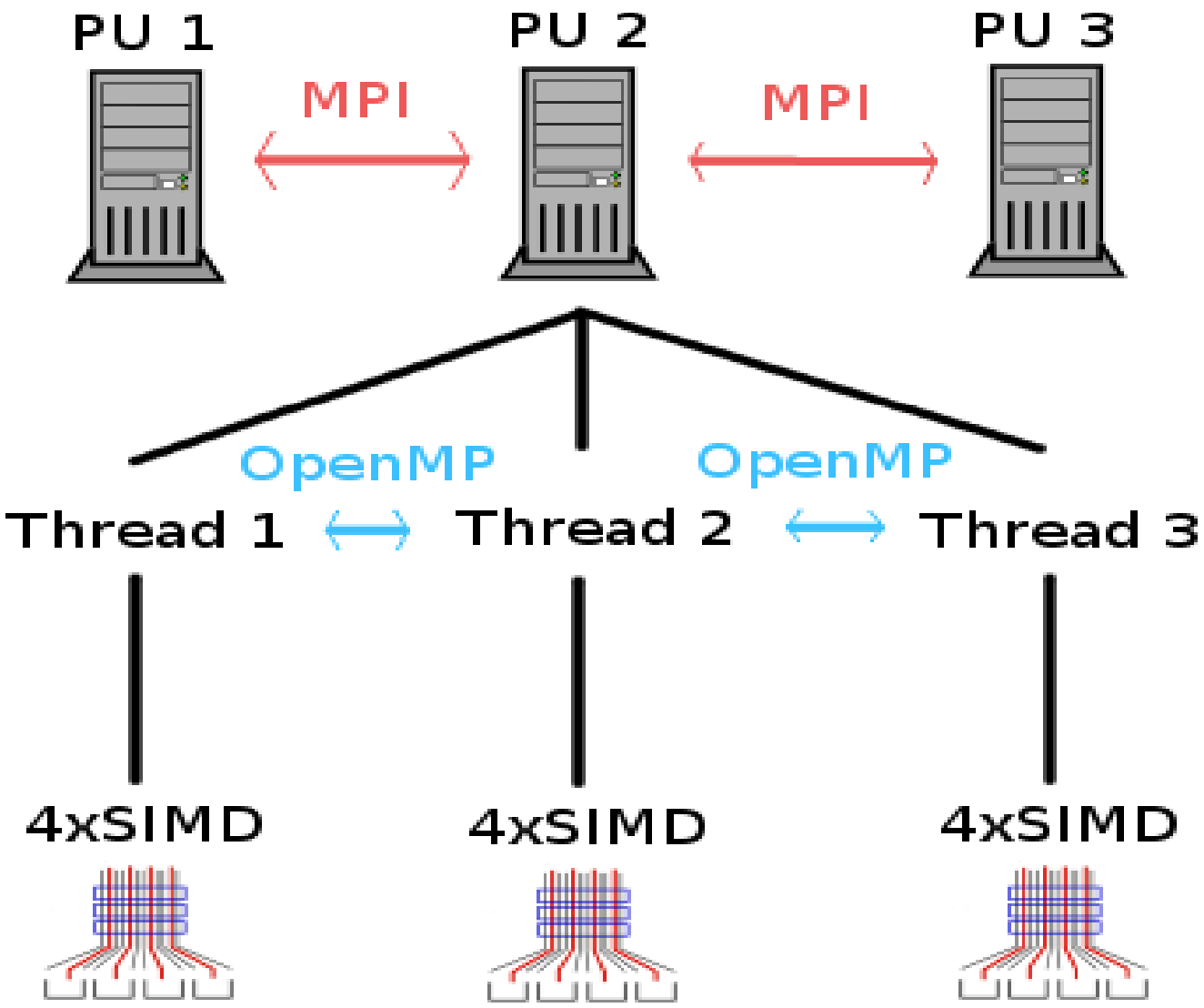
Sparsh Mittal

Courtesy for some slides: S. R. Sarangi and others

Acronyms

- PU = Processing Unit
- SIMD = Single Instruction Multiple Data
- MPI = Message Passing Interface
- OpenMP = Open Multi Processing

A parallel program running on three computation units with three threads each with width 4 SIMD unit in each thread



http://sci.tuomastonteri.fi/programming/sse/2

Multiprocessing (by Message Passing)

Multiprocessing (by Message Passing)

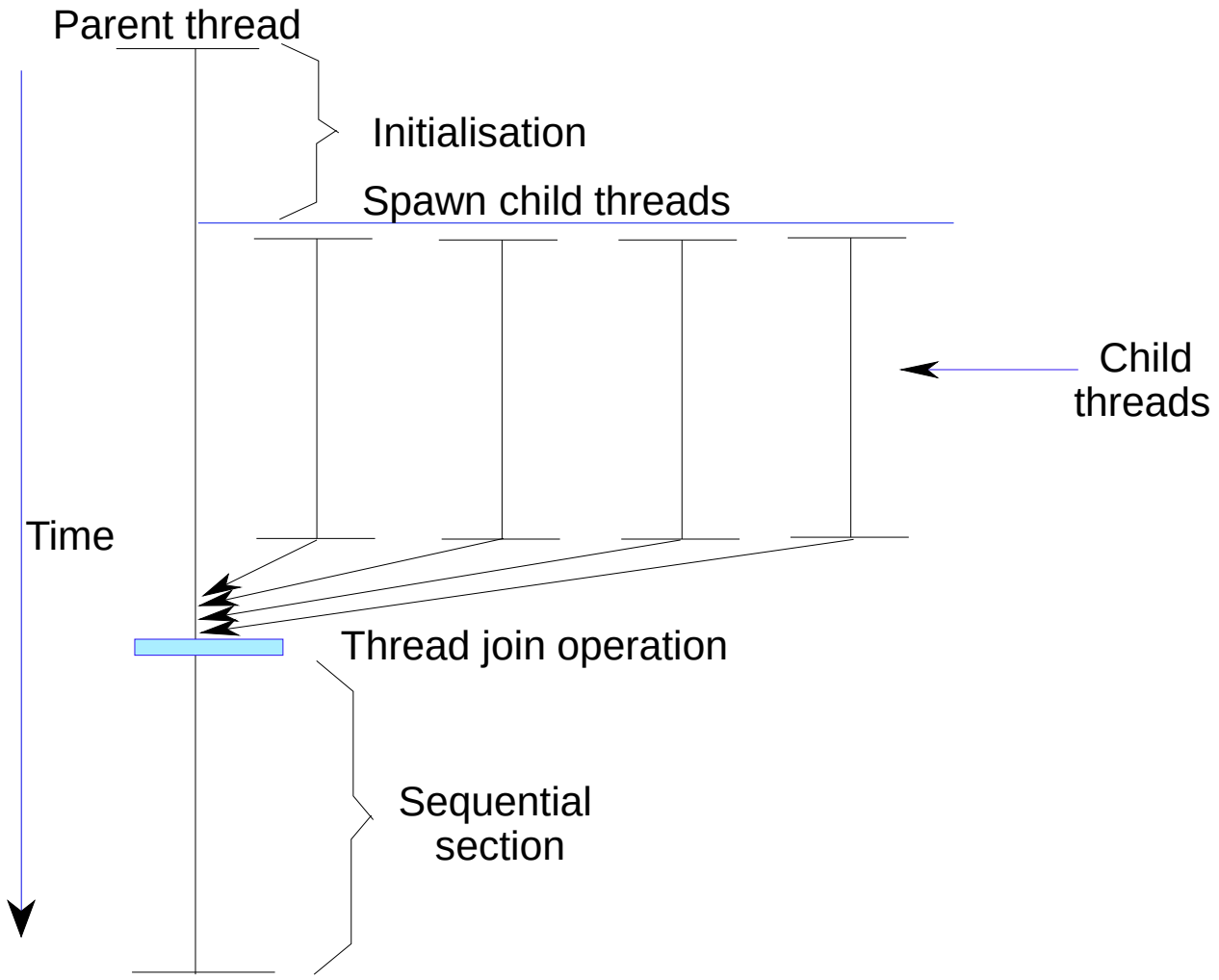
- Multiprocessing term is loosely defined.
- Here, we use it to mean "simultaneously using multiple processes, running on different processing units”.
- In message passing, programs communicate between each other by sending and receiving messages (e.g., sending emails).
- They do not share memory addresses.
  - Example language: MPI

Multithreading

The Notion of Threads

* We spawn a set of separate threads in each process
* Properties of threads
  * It has its own program counter, set of registers, and stack.
  * A thread shares its address space with other threads
    - By contrast, in message passing, they do not share their address space.
  * Threads communicate with each other by writing values to memory or via synchronization operations.

Operation of the Program



Multithreading

* Multithreading runs multiple threads on the same pipeline
* Three types
  * Coarse grained
  * Fine grained
  * Simultaneous

Analogy

- Consider a car which can be shared by 4 people (A, B, C, D) in following way:
  - Option1: Four people ride the car simultaneously almost all the time to go to their destinations.
  - Option2: A uses the car for 15 days, B for 14 days, C for 18 days and D for 16 days (and repeat).
  - Option3: A uses the car for 8 hours, B for 9 hours, C for 5 hours and D for 7 hours (and repeat).
- Match the above three options to three types of multithreading.

Coarse-Grained Multithreading

- Assume we have 4 threads
- We can have an **algorithm** that switches
  - every n cycles, or
  - when there is a **long latency event** such as an L2 cache miss. For example, when thread 1 has an L2 cache miss, switch to thread 2. When thread 2 has an L2 miss, switch to thread 3.
- Minimizes idle cycles for the entire system.

Fine-Grained Multithreading

- The switching granularity is very small.
  - 1-2 cycles
- **Advantage** :
  - Can take advantage of low latency events such as division, or L1 cache misses.
  - Minimize idle cycles to an even greater extent.

Events and their latency

Q Write one example each of low latency and high latency events

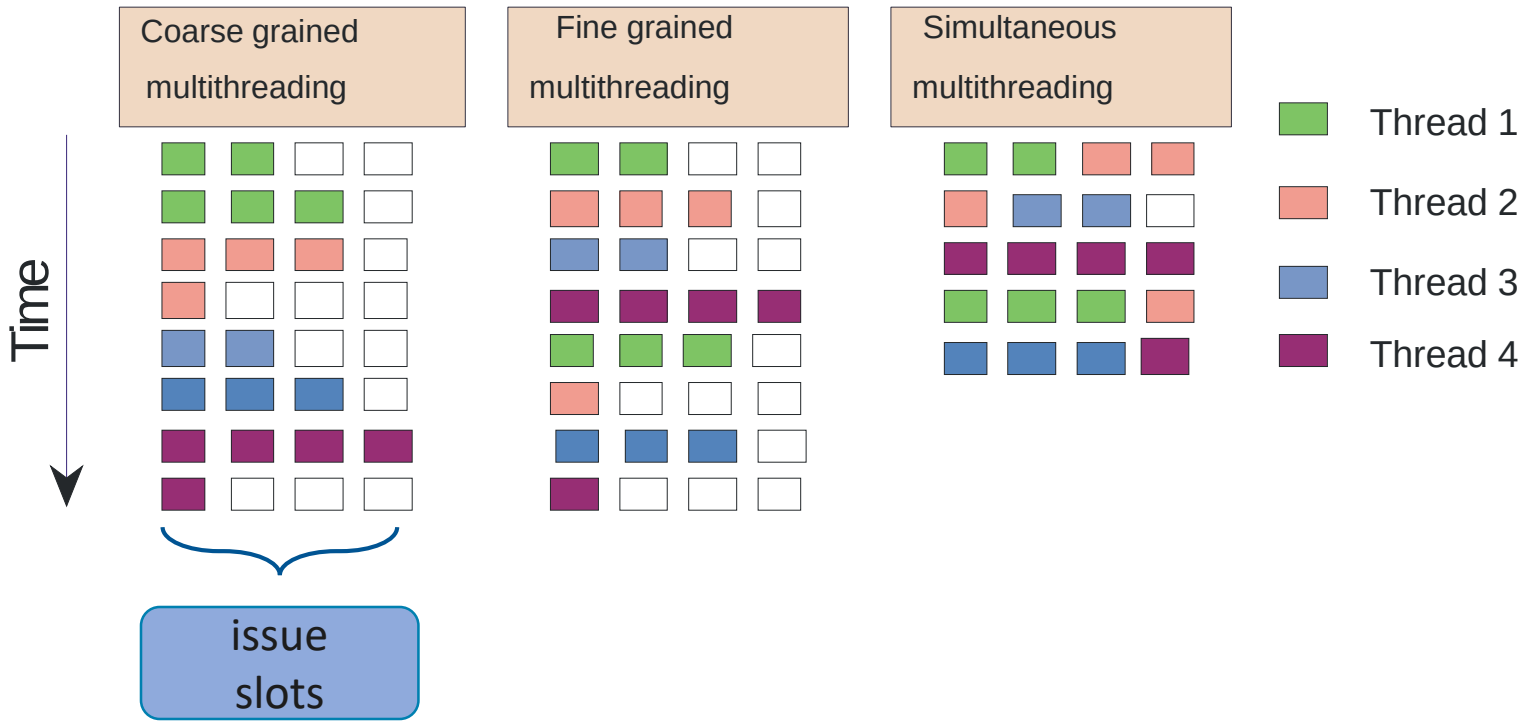
Low-latency: L1 cache miss or division.

High-latency: L2 cache miss (main memory access)

Simultaneous Multithreading

- We simultaneously issue multiple instructions to the functional units.
- For example, a 3 issue processor can fetch, decode, and execute 3 instructions per cycle.

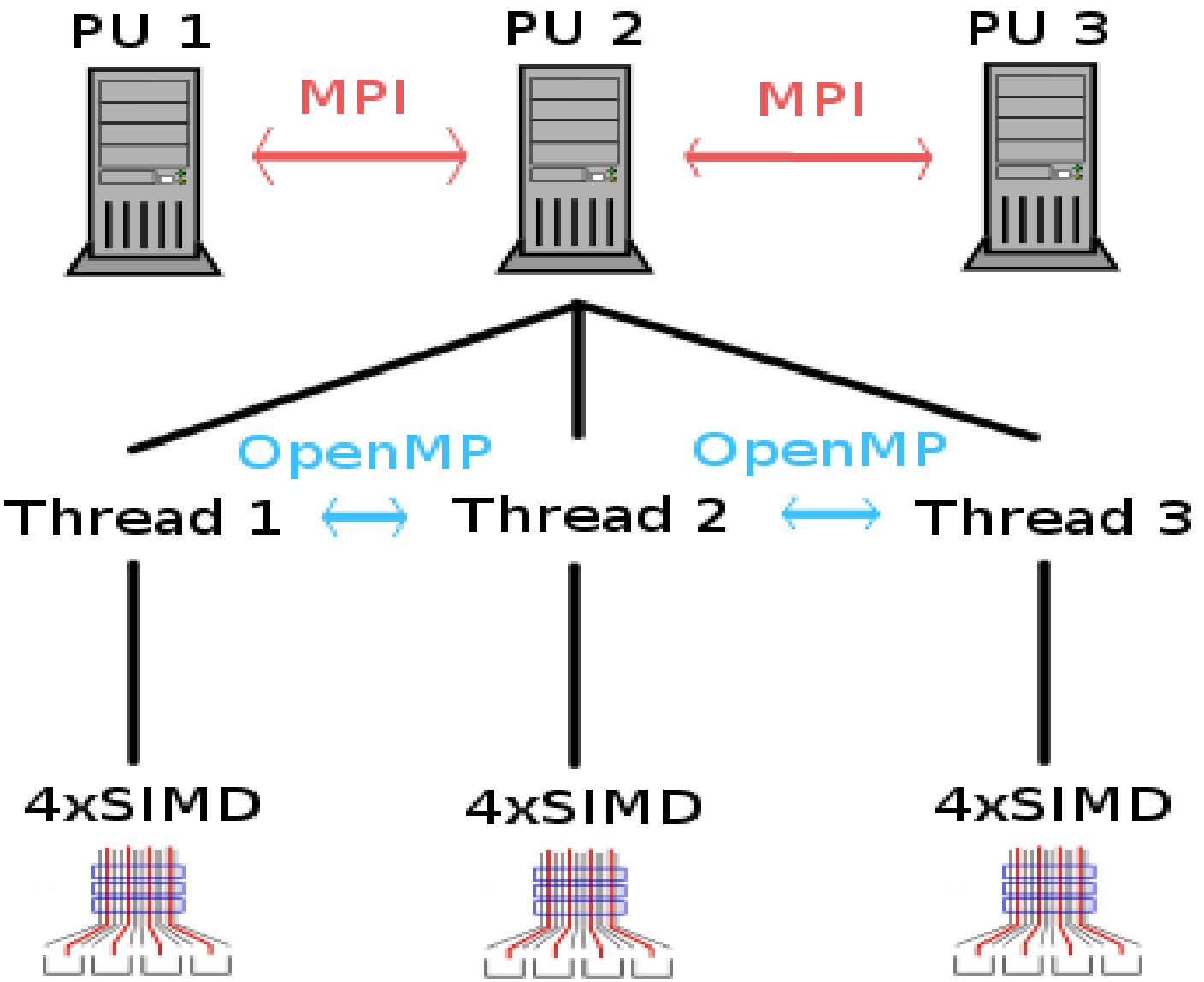
Summary



Courtesy : S. R. Sarangi

SIMD

A parallel program running on three computation units with three threads each with width 4 SIMD unit in each thread

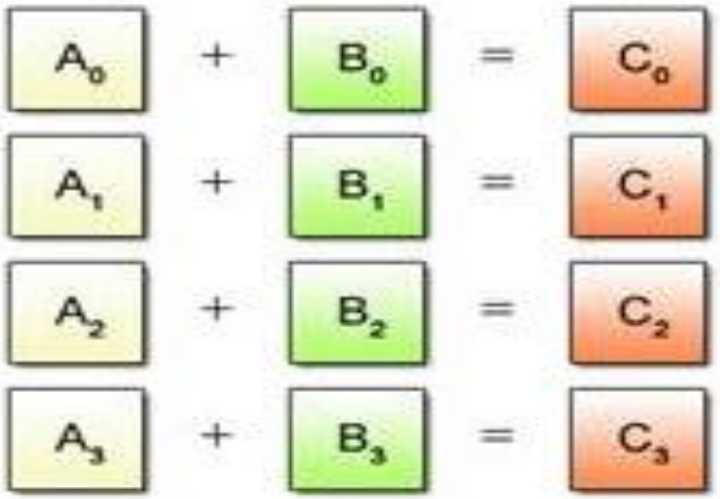


BIG PICTURE

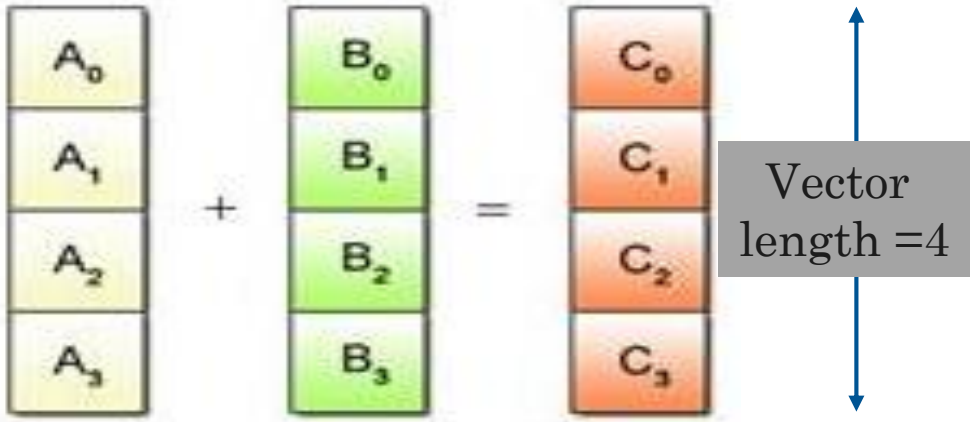
http://sci.tuomastonteri.fi/programming/sse/2

SIMD

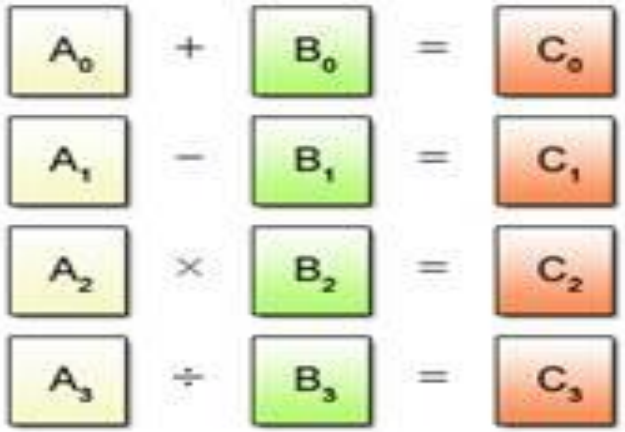
(a) Scalar Operation



(b) SIMD Operation (vectorization)

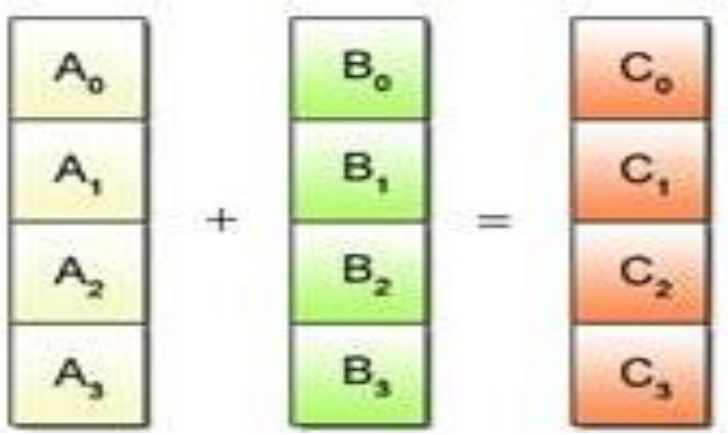


(c) Not allowed under SIMD (different operations on different data)



ftp.cvut.cz/kernel/people/geoff/cell/ps3-linux-docs/CellProgrammingTutorial/BasicsOfSIMDProgramming.html

More insights on SIMD operation



Vector length (or SIMD width) =4
If each scalar operand is 32b,
then, SIMD register size is 128b

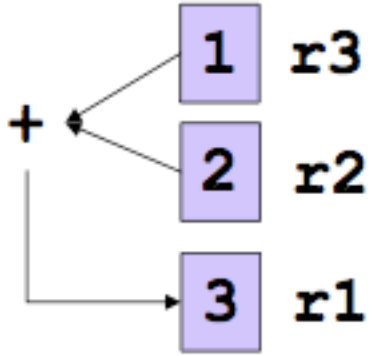
Addition of vector registers is equivalent to pairwise addition of individual elements.

In above example, a single vector instruction can replace 4 scalar instructions.

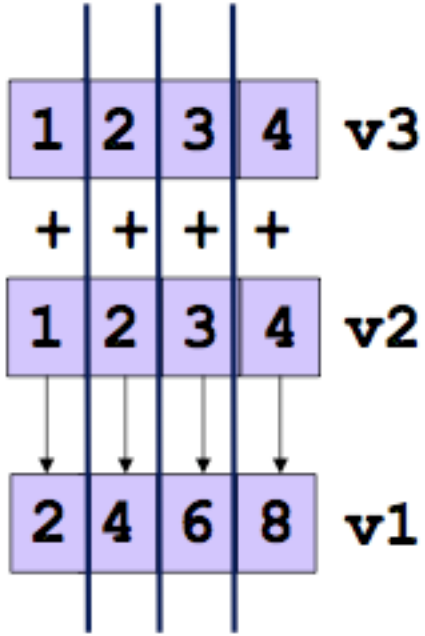
http://ftp.cvut.cz/kernel/people/geoff/cell/ps3-linux-docs/CellProgrammingTutorial/BasicsOfSIMDProgramming.html

Scalar vs SIMD

```
Scalar: add r1,r2,r3
```




```
SIMD: vadd<sws> v1,v2,v3
```



Some SIMD Instruction Sets Used in Industry

SIMD Register size	Instruction set
80 bits	MMX
128-bits	SSE1/SSE2 etc.
256-bits	AVX/AVX2
512-bits	AVX-512

Solved example

Assume that we have a vector register of width 16 bits and the width of each element is 8 bits. VReg1 and VReg2 are 4ABC and D8AF, respectively. Find the value of vector register obtained by adding these two vector registers. Any carry-outs during addition are ignored.

4A BC
+D8 AF

Answer: BC is 10111100 and AF is 10101111. On adding them, we get 101101011, where only 8 bits are kept and carry-out is ignored.

4A is 01001010 and D8 is 11011000. On adding them, we get 100100010, where only 8 bits are kept and carry-out is ignored.

So, final answer is 00100010 01101011, which is 226B.

References

- S. Mittal et al, “A Survey on Evaluating and Optimizing Performance of Intel Xeon Phi” 2019 ( )
- http://ftp.cvut.cz/kernel/people/geoff/cell/ps3-linux-docs/CellProgrammingTutorial/BasicsOfSIMDProgramming.html
- https://www.intel.com/content/www/us/en/products/sku/237247/intel-xeon-platinum-8571n-processor-300m-cache-2-40-ghz/specifications.html
- https://stackoverflow.blog/2020/07/08/improving-performance-with-simd-intrinsics-in-three-use-cases/
- http://const.me/articles/simd/simd.pdf